

Choose a design pattern for your agentic AI system

This document provides guidance to help you choose a design pattern for your agentic AI system. *Agent design patterns* are common architectural approaches to build agentic applications. An agent design pattern offers a distinct framework for organizing a system's components, integrating the model, and orchestrating a single agent or multiple agents to accomplish a workflow.

AI agents are effective for applications that solve open-ended problems, which might require autonomous decision-making and complex multi-step workflow management. Agents excel at solving problems in real-time by using external data and they excel at automating knowledge-intensive tasks. AI agents are suitable when you need AI to complete goal-focused tasks with some degree of autonomy. For other use cases, you can use assistive and generative AI applications.

This guide assumes that you have a foundational knowledge of agentic AI systems and how their architecture differs from that of non-agentic systems, such as those that use direct model reasoning or retrieval-augmented generation (RAG).

Overview of the design process

The following are the high-level steps to choose a design pattern for your agentic AI system:

- 1 **Define your requirements:** Assess the characteristics of your workload, including task complexity, latency and performance expectations, cost budget, and the need for human involvement.
- 2 **Review the common agent design patterns:** Learn about the common design patterns in this guide, which include both single-agent systems and multi-agent systems.
- 3 **Select a pattern:** Select the appropriate design pattern based on your workload characteristics.

This process isn't a one-time decision. You should periodically revisit these steps to refine your architecture as your workload characteristics change, your requirements evolve, or new Google Cloud features become available.

Define your requirements

The questions that follow aren't exhaustive checklists for planning. Use these questions as a starting point to identify the primary goal of your agentic system and to select the best design pattern.

- bullet **Task characteristics:** Can your task be completed in predefined workflow steps or is the task open-ended? Does your task need to use an AI model to orchestrate the workflow?
- bullet **Latency and performance:** Do you need to prioritize fast or interactive responses at the cost of accuracy or high-quality responses? Or can your application tolerate a delay to achieve a more accurate or thorough result?
- bullet **Cost:** What is your budget for inference costs? Can you support patterns that require multiple calls to the model for a single request?

bullet Human involvement: Does your task involve high-stakes decisions, safety-critical operations, or subjective approvals that require human judgment?

If your workload is predictable or highly structured, or if it can be executed with a single call to an AI model, it can be more cost effective to explore non-agentic solutions. For example, you might not need an agentic workflow for tasks like summarizing a document, translating text, or classifying customer feedback.

Single-agent system

A *single-agent system* uses an AI model, a defined set of tools, and a comprehensive system prompt to autonomously handle a user request or to complete a specific task. In this fundamental pattern, the agent relies on the model's reasoning capabilities to interpret a user's request, plan a sequence of steps, and decide which tools to use from a defined set. The system prompt shapes the agent's behavior by defining its core task, persona, and operations, and the specific conditions for using each tool.

A single-agent system is ideal for tasks that require multiple steps and access to external data. For example, a customer support agent must query a database to find an order status, or a research assistant needs to call APIs to summarize recent news. A non-agentic system can't perform these tasks because it can't autonomously use tools or execute a multi-step plan to synthesize a final answer.

If you're early in your agent development, start with a single agent. A single agent's performance can become less effective as the number of tools and task complexity grow — this can show up as increased latency, incorrect tool use, or failure to complete the task. Refining the agent's reasoning with techniques like the ReAct pattern often mitigates these issues. For workflows requiring several distinct responsibilities, consider a multi-agent system.

Multi-agent systems

A *multi-agent system* orchestrates multiple specialized agents to solve a complex problem that a single agent can't easily manage. The core principle is to decompose a large objective into smaller sub-tasks and assign each sub-task to a dedicated agent with a specific skill. These agents then interact through collaborative or hierarchical workflows to achieve the final goal. Multi-agent patterns provide a modular design that can improve scalability, reliability, and maintainability compared to a single agent with a monolithic prompt.

Each agent in a multi-agent system requires a specific context to perform its task — documentation, historical preferences, relevant links, conversation history, or operational constraints. The process of managing this information flow is called *context engineering*. It includes strategies like isolating context for a specific agent, persisting information across multiple steps, or compressing large amounts of data.

Building a multi-agent system requires additional evaluation, security, reliability, and cost considerations versus a single-agent system: precise access controls per agent, robust orchestration for inter-agent communication, and managing the increased operational cost of running multiple agents.

Sequential pattern

The *multi-agent sequential pattern* executes a series of specialized agents in a predefined linear order — the output of one agent becomes the direct input of the next. The orchestrator is a workflow agent that

operates on predefined logic without consulting an AI model to route work.

Use the sequential pattern for highly structured, repeatable processes where the sequence of operations doesn't change. For example, a data pipeline could chain a data extraction agent → a cleaning agent → a loading agent. The pattern reduces latency and cost compared to model-orchestrated patterns, but its rigid structure makes it hard to adapt or skip steps.

Parallel pattern

The *multi-agent parallel pattern* (also called the *concurrent pattern*) runs multiple specialized subagents at the same time on independent sub-tasks; their outputs are then synthesized into a final consolidated response. Like the sequential pattern, it uses a workflow agent rather than a model to manage execution.

Use the parallel pattern when sub-tasks can run concurrently — e.g., to analyze customer feedback, fan out a single entry to a sentiment agent, keyword agent, categorization agent, and urgency agent simultaneously, then gather. It reduces wall-clock latency but increases immediate resource use and token consumption, and requires gather logic to reconcile potentially conflicting results.

Loop pattern

The *multi-agent loop agent pattern* repeatedly executes a sequence of specialized subagents until a specific termination condition is met. The loop agent operates on predefined logic and evaluates the exit condition (a maximum number of iterations, or a custom state) after each iteration.

Use it for tasks requiring iterative refinement or self-correction. Its main risk is an infinite loop: if the termination condition is poorly defined or unreachable, the loop can run indefinitely, driving up costs and resource consumption.

Review and critique pattern

The *multi-agent review and critique pattern* (also called the *generator and critic pattern*) improves quality and reliability by using two specialized agents. A generator produces an initial output; a critic evaluates it against predefined criteria — factual accuracy, formatting rules, safety guidelines — and can approve, reject, or return it for revision. It's an implementation of the loop pattern.

Use it where outputs must be highly accurate or meet strict constraints. For example, in code generation, a generator writes a function and a critic acts as a security auditor, scanning for vulnerabilities and verifying that unit tests pass before approval. The quality gain costs at least one extra model call per cycle, and revision loops accumulate latency and cost.

Iterative refinement pattern

The *iterative refinement pattern* uses a looping mechanism to progressively improve an output. One or more agents modify a result stored in session state during each iteration until the output meets a quality threshold or reaches a max iteration count.

It suits complex generation tasks hard to achieve in one step — writing and debugging code, developing a detailed multi-part plan, or drafting a long-form document. A creative-writing agent might generate a draft, critique it for flow and tone, and rewrite based on the critique. Each loop adds latency and cost, so carefully designed exit conditions are essential.

Coordinator pattern

The *multi-agent coordinator pattern* uses a central *coordinator* agent that analyzes and decomposes the user's request, then dispatches each sub-task to a specialized agent. Unlike the parallel pattern, the coordinator uses an AI model to orchestrate and dynamically route tasks.

Use it for structured business processes that need adaptive routing. A customer service coordinator could classify an incoming message as an order-status request, return, or refund, and route it accordingly. The flexibility costs more model calls — both at the coordinator and at each specialist — which raises tokens, cost, and end-to-end latency.

Hierarchical task decomposition pattern

The *hierarchical task decomposition pattern* organizes agents in a multi-level hierarchy. A top-level *root* agent receives a complex task and decomposes it into manageable sub-tasks, delegating each to a specialized subagent. This can repeat through multiple layers until tasks are simple enough for a worker agent to execute. It's an implementation of the coordinator pattern.

Use it for ambiguous, open-ended problems requiring multi-step reasoning — research, planning, and synthesis. A research project coordinator might delegate data gathering, analysis, and report writing to specialist subagents. Multi-level structure brings architectural complexity and many model calls, raising latency and cost significantly.

Swarm pattern

The *multi-agent swarm pattern* uses collaborative, all-to-all communication among specialized agents to iteratively refine a solution. A dispatcher agent routes the request and facilitates communication, but doesn't orchestrate — each agent can communicate with every other, share findings, critique proposals, and hand off the task.

Because there's no central supervisor, you must define an explicit exit condition — max iterations, a time limit, or reaching consensus — to ensure the swarm halts. Use it for ambiguous, highly complex problems that benefit from debate (e.g., a market researcher, engineering agent, and financial modeling agent collectively designing a new product). It's the most complex and costly multi-agent pattern, with significant risk of unproductive loops.

Reason and act (ReAct) pattern

The *ReAct pattern* uses the AI model to frame its thought process and actions as a sequence of natural language interactions. The agent operates in an iterative loop of thought, action, and observation until an exit condition is met:

- bullet **Thought:** The model reasons about the task and decides what to do next, evaluating whether the user's request has been fully answered.
- bullet **Action:** If the task isn't complete, the model selects a tool and forms a query to gather more information. If it is complete, it formulates the final answer to send to the user.
- bullet **Observation:** The model receives the tool's output and saves relevant information in its memory, building on previous observations to avoid repetition or losing context.

The loop terminates when the agent finds a conclusive answer, reaches a preset maximum iteration count, or hits an error. Use it for complex, dynamic tasks requiring continuous planning and adaptation — for example, a robotics agent that calculates a path, moves, observes the environment, and replans around new obstacles.

A single ReAct agent can be simpler and cheaper than a complex multi-agent system, and its thinking transcript helps with debugging. But the iterative loop typically increases end-to-end latency, and the agent's effectiveness depends heavily on the model's reasoning — a single bad tool result can propagate through the loop.

Human-in-the-loop pattern

The human-in-the-loop pattern integrates points for human intervention directly into the agent's workflow. At a predefined checkpoint the agent pauses, calling an external system to wait for a person to review its work — approving a decision, correcting an error, or providing necessary input before the agent can continue.

Use it for high-stakes, safety-critical, or subjective tasks: approving a large financial transaction, validating the summary of a sensitive document, or providing subjective feedback on creative content. For example, an agent that anonymizes a patient dataset might automatically redact protected health information, then pause for a human compliance officer to validate before release. The pattern adds safety and reliability at the cost of building and maintaining the external system for user interaction.

Custom logic pattern

The custom logic pattern provides maximum flexibility. You implement orchestration in code — conditional statements, complex workflows with multiple branching paths — when no standard pattern fits.

As an example, consider a customer refund workflow:

- 1 The user sends a query to the customer refund agent, which acts as a coordinator.
- 2 The coordinator's custom logic invokes a parallel verifier that dispatches a purchaser-verifier agent and a refund-eligibility agent simultaneously.
- 3 Once results are gathered, the coordinator runs a tool to check whether the request is eligible for a refund.
- 4 If eligible, the coordinator routes to a refund-processor agent that calls the `process_refund` tool. If not eligible, it routes to a sequential flow starting with a store-credit agent and a process-credit-decision agent.
- 5 The result from whichever path is taken is sent to a final-response agent that formulates the answer for the user.

Use custom logic when you need fine-grained control over execution, or when the workflow doesn't fit any other pattern. You're responsible for designing, implementing, and debugging the entire orchestration flow, which is more error-prone than using a predefined pattern.

Compare design patterns

Choosing an agent pattern is a fundamental architectural decision. Each pattern offers different trade-offs in flexibility, complexity, and performance.

Workflows that are deterministic

Tasks are predictable and sequential, with a clearly defined workflow path. The steps are known in advance and the process doesn't change much from run to run.

Workload characteristics	Agent design pattern
• Multi-step tasks that follow a predefined, rigid workflow. • Doesn't require model orchestration. • Fixed sequence — output of one agent is the direct input of the next.	Multi-agent sequential pattern
• Independent tasks that can be executed at the same time. • Doesn't require model orchestration. • Reduces overall latency by running sub-tasks simultaneously.	Multi-agent parallel pattern
• Open-ended or complex generation tasks hard to complete in one attempt. • Requires progressively improving the output over multiple cycles. • Doesn't require model orchestration. • Prioritizes output quality over latency.	Multi-agent iterative refinement pattern

Workflows that require dynamic orchestration

Complex problems where agents must determine how to proceed. The system needs to dynamically plan, delegate, and coordinate tasks without a predefined script.

Workload characteristics	Agent design pattern
• Structured, multi-step tasks that require external tools. • Requires fast development for a prototype / proof of concept.	Single agent pattern
• Dynamic routing to a specialized subagent for varied input. • Higher latency from multiple coordinator model calls. • Higher cost from multiple coordinator calls.	Multi-agent coordinator pattern
• Multi-level model orchestration for complex, open-ended, ambiguous tasks. • Requires comprehensive, high-quality results where decomposing ambiguity is the main challenge. • High latency from nested, multi-level decomposition.	Multi-agent hierarchical task decomposition pattern

Workload characteristics	Agent design pattern
• Collaborative debate and iterative refinement for highly complex, ambiguous tasks. • Prioritizes synthesis of multiple perspectives for comprehensive or creative solutions. • High latency and cost from dynamic, all-to-all agent communication.	Multi-agent swarm pattern

Workflows that involve iteration

Tasks where the final output is achieved through cycles of refinement, feedback, and improvement.

Workload characteristics	Agent design pattern
• Agent must iteratively reason, act, and observe to build or adapt a plan. • Prioritizes a more accurate and thorough result over latency.	ReAct pattern
• Monitoring or polling tasks repeating a predefined action until an exit condition. • Unpredictable or long-running latency while waiting for exit condition.	Multi-agent loop pattern
• Tasks require a distinct validation step before completion.	Multi-agent review and critique pattern
• Open-ended or complex generation tasks hard to complete in one attempt. • Requires progressively improving the output over multiple cycles. • Doesn't require model orchestration. • Prioritizes output quality over latency.	Multi-agent iterative refinement pattern

Workflows that have special requirements

Tasks that don't follow common agentic patterns — unique business logic, or require human judgment and intervention at critical points.

Workload characteristics	Agent design pattern
• Requires human supervision for high-stakes or subjective tasks (safety, reliability, compliance).	Human-in-the-loop pattern

Workload characteristics	Agent design pattern
• Complex, branching logic beyond a direct linear sequence. • Requires maximum control mixing predefined rules with model reasoning. • Fine-grained process control for workflows that don't fit a standard template.	Custom logic pattern